PRE-LAUNCH SECURITY AND QUALITY AUDIT

Security and Quality Audit

Chairside - salon and barbershop booking app

Findings, business impact, and remediation

October 2026

Sample report - published as a portfolio example

Prepared by Atıl Arslan

Executive summary

This report reviews a salon and barbershop booking app for security and correctness, and documents the fixes applied. It is written for the business owner: each finding explains what was found, why it matters for the business, and what was done about it.

0	2	11	5	1
CRITICAL	HIGH	MEDIUM	LOW	INFORMATIONAL

The app is built on solid foundations. The database keeps each salon's data separate, visitors cannot read the tables directly, and the system already prevents two bookings landing on the same person at the same time. The automated security scan offered by the build platform reported no issues at the time of the audit (October 2026); that same report notes it cannot catch every possible risk.

The issues found are subtler than missing basics: the booking and permission rules were enforced only in the app's screens, not on the server. Anyone who talks to the system directly, bypassing the screens, could do things the screens never allow. That is the thread running through the most serious findings.

> Of 19 findings, the 2 High and most Medium issues are fixed or materially reduced; the rest are documented with clear recommendations. Every fix is backed by an automated test that failed before the change and passes after it. The original app is preserved unchanged for before-and-after comparison.

THE BIGGEST RISKS IN PLAIN TERMS

- A stranger who knew a client's phone number could change that client's name and email.
- A single visitor could flood the booking calendar and client list with fake entries and make it unusable.
- Any staff member could delete a client and that client's entire history.
- Bookings could be created at closed times (03:00, closed days, or years ahead) by bypassing the app.
- Typing someone's email into the staff list could silently give them access to your client data.

Latest test run (2026-10-12): 42 tests, 41 passed, 1 skipped (an environment-dependent time-zone demonstration).

How severity is rated

Every finding is rated on the same short scale, judged by who could exploit it and what it would affect:

LEVEL	WHAT IT MEANS
Critical	An anonymous, unauthenticated attacker can read or alter data across salons, or take over accounts, with little effort.
High	An anonymous or low-privilege caller, by talking to the system directly, can read or change another party's personal data, or deny a core business function such as the booking calendar - with no insider help.
Medium	Needs a valid account, a specific piece of knowledge, or an insider step; or the damage is recoverable; or it is an extra layer of defence that only matters if another flaw is also present.
Low	Limited or hard-to-reach impact, or a good-practice, hygiene or maintainability gap.
Informational	No practical security impact; recorded for completeness.

What was done well

Not everything needed fixing. These are the parts the app already got right:

- Each salon's data is isolated from every other salon's (verified by tests).
- Anonymous visitors cannot read database tables directly; public actions go only through a small set of controlled functions.
- Double-booking the same staff member at the same time is prevented by the database itself.
- Prices on past bookings are snapshotted, so later price changes do not rewrite history.
- The powerful server key never reaches the browser.
- The AI features run as the signed-in user, so they cannot reach another salon's clients.

ADDITIONAL HARDENING APPLIED

- Anonymous visitors had database-level table privileges by default, leaving the access rules as the only line of defence; those privileges were removed, so anonymous visitors can now use only the controlled public functions.
- Phone numbers are normalised to one format so the same client is not stored twice.

Design decisions (intentional behaviour)

Some behaviour that an automated scan might flag is deliberate. These are choices the salon owner made, not defects:

- Owners and staff can place, move or cancel appointments outside published opening hours and on closed days. This is intentional: salons routinely take off-hours or day-off appointments, so the owner keeps full control of their own calendar. The public booking page that clients use is still held to the opening-hours, open-day and 21-day advance rules (see BH-01), and the system still prevents two appointments from overlapping on any path.

SEC-01 High — Partially fixed

A visitor could change an existing client's name and email

WHAT WAS FOUND

The public booking flow identified clients by phone number. A booking made with an existing client's phone number overwrote that client's stored name and email with whatever was typed.

WHY IT MATTERS FOR THE BUSINESS

Anyone who knew (or guessed) a client's phone number could rename that client and change their email, and the new booking attached to that client's history and notes. This corrupts the client list and could be used to impersonate or harass a client.

WHAT WE CHANGED

We changed the rule that saves a client from a public booking so it never overwrites a client who already exists: it keeps the stored name and email and only fills in blanks. We also added a field that records the name typed at booking time and show it to staff on the client screen when it differs, so they can check it is the same person.

WHERE

Files: `drizzle/migrations/0001_audit_hardening.sql`, `src/routes/_authenticated/customers.$id.tsx`

Commit: 17f8c81, 255d78d

HOW WE VERIFIED IT

Test '[#1]' in public-booking.test.ts and http-e2e.test.ts failed on the original app (the client's name was overwritten) and passes after the change (the stored name is unchanged).

WHAT REMAINS

A stranger who knows a client's phone number can still attach a booking to that client's record without changing it. Closing this fully needs a one-time code sent to the phone at booking time.

BH-02 High

Partially fixed

No limit on public bookings

WHAT WAS FOUND

A single anonymous visitor could create an unlimited number of bookings; there was no rate limit or quota.

WHY IT MATTERS FOR THE BUSINESS

An attacker could fill the calendar and client table with fake entries and make the system unusable for the salon.

WHAT WE CHANGED

We added two limits to the booking function: at most 5 upcoming bookings per phone number, and at most 10 bookings per salon per minute.

WHERE

Files: `drizzle/migrations/0001_audit_hardening.sql`

Commit: 408b033

HOW WE VERIFIED IT

Two tests in public-booking.test.ts (the per-phone cap and the per-salon burst) accepted the flood before and stop it after.

WHAT REMAINS

Per-visitor rate limiting and a CAPTCHA need a layer in front of the app (the booking call comes straight from the browser), and the AI features still need a per-user usage limit.

BH-01 Medium Fixed

Bookings could be made at impossible times

WHAT WAS FOUND

The rules about opening hours, open days and how far ahead clients may book existed only in the app's screens. Calling the booking service directly, the only server check was that the time was not in the past.

WHY IT MATTERS FOR THE BUSINESS

Someone bypassing the app could create appointments while the salon is closed, outside opening hours, or years in the future. The public calendar stops reflecting reality and staff cannot trust it.

> **Design decision:** These rules apply to the public booking page only. When the owner or staff create or move a booking inside the signed-in app, they are deliberately allowed to book outside opening hours and on closed days (see Design decisions). Overlap is still prevented on every path.

WHAT WE CHANGED

We added the booking rules to the server function that creates a public booking: it now checks the salon's open days, opening hours, a 21-day limit, and that the chosen time lands on a valid 15-minute slot, all judged in the salon's own time zone. We also added a time-zone setting to each salon so the checks use the right clock.

WHERE

Files: `drizzle/migrations/0001_audit_hardening.sql`

Commit: 28d4163

HOW WE VERIFIED IT

Tests '[#2] 03:00', '[#2] years in the future', '[#2] closed day' and '[BH-01] 15-minute grid' in public-booking.test.ts were accepted on the original app and are rejected after the change.

SEC-03 Medium — Fixed

Staff could do more than their role should allow

WHAT WAS FOUND

Any staff member could edit and delete client records, and could read every booking in the salon. Deleting a client also deleted that client's entire booking and note history.

WHY IT MATTERS FOR THE BUSINESS

A single stylist could wipe a client's whole history, or browse other staff members' clients - well beyond 'see your own calendar'.

WHAT WE CHANGED

We changed the database access rules: staff may read clients (needed to keep notes), but only the owner may add, change or delete them; and each staff member can read only the bookings assigned to them, while the owner reads all.

WHERE

Files: `drizzle/migrations/0001_audit_hardening.sql`

Commit: 3c08384

HOW WE VERIFIED IT

Tests in role-matrix.test.ts and http-e2e.test.ts show staff can no longer edit or delete clients, and a test in pentest.test.ts shows a staff member sees only their own bookings. All failed before and pass after.

WHAT REMAINS

Staff can still read the salon's whole client list (names, phone, email, notes), because they need it to keep notes; only their booking view is limited to their own. Restricting the client list per staff member would be a separate product decision. Owners can also still delete a client (which removes that client's history); a soft-delete that hides instead of erasing is recommended.

SEC-04 Medium

Fixed

Staff invitations joined people without consent

WHAT WAS FOUND

Adding an email address to the staff list silently linked that person to the salon the next time they signed in, with no acceptance step.

WHY IT MATTERS FOR THE BUSINESS

A person could be given access to a salon's client data without ever agreeing to join, and the behaviour was unpredictable when an email belonged to more than one salon.

WHAT WE CHANGED

We removed the automatic linking on sign-in and added an explicit invitation: the invited person sees a pending invite and must click Accept, and only an invite addressed to their own email can be accepted.

WHERE

Files: `drizzle/migrations/0001_audit_hardening.sql`, `src/hooks/use-me.ts`, `src/components/app-shell.tsx`

Commit: 2ade4e1

HOW WE VERIFIED IT

Tests in pentest.test.ts confirm sign-in no longer auto-joins, a different email cannot accept the invite, and the invited person can accept their own.

SEC-02 Medium

Fixed

No limits on booking form input

WHAT WAS FOUND

The booking form accepted names, phone numbers and emails of unlimited length and with no format checking (a 20,000-character name was accepted).

WHY IT MATTERS FOR THE BUSINESS

Oversized or malformed data could break staff screens and outgoing messages and bloat the database.

WHAT WE CHANGED

We added length and basic format checks to the booking function for the name, phone number and email.

WHERE

Files: `drizzle/migrations/0001_audit_hardening.sql`

Commit: 28d4163

HOW WE VERIFIED IT

Test '[#2/SEC-02] name length' in public-booking.test.ts accepted a 20,000-character name before and rejects it after.

SEC-05 Medium

Fixed

Missing security headers

WHAT WAS FOUND

The app sent none of the standard browser security headers (such as a content security policy or anti-framing header).

WHY IT MATTERS FOR THE BUSINESS

The site could be embedded invisibly in a malicious page to trick clients (clickjacking), and browser defences against injected scripts were weaker than they should be.

WHAT WE CHANGED

We added a content security policy and the other standard security headers to every server response, and allowed the specific resources the app actually needs so nothing breaks.

WHERE

Files: `src/lib/security-headers.ts`, `src/server.ts`

Commit: 2865255

HOW WE VERIFIED IT

Test in security-headers.test.ts checks the headers are present, and a live check confirmed the booking and sign-in pages load with no policy violations.

WHAT REMAINS

The policy still permits inline scripts (it has to, because the app's framework injects them as the page loads), which weakens its protection against injected scripts. Tightening it to a stricter, nonce-based policy is a recommended follow-up (see the roadmap).

SEC-06 Medium

Recommendation

Login session stored in the browser

WHAT WAS FOUND

The login session token is kept in the browser's local storage.

WHY IT MATTERS FOR THE BUSINESS

If a script-injection flaw ever occurred, an attacker could read the session and take over the account.

HOW TO FIX

1. Store the login session in a secure, HttpOnly cookie so page scripts cannot read it.
2. Keep the content security policy added in SEC-05 in place as an added layer.

SEC-10 Medium

Partially fixed

Personal data handling and the AI features

WHAT WAS FOUND

Client personal data (name, phone, email, notes, visit history) is sent to an outside AI provider for the summary and reminder features. There was no stated retention period, no deletion or export path, and no privacy notice on the booking page.

WHY IT MATTERS FOR THE BUSINESS

Clients' personal data leaves the salon's system with no stated handling, and the salon cannot easily honour a request to delete someone's data.

WHAT WE CHANGED

So far only one thing has changed: a short privacy notice was added to the public booking page, telling clients their details are shared only with the salon and how to ask for changes or deletion. The substantive data-protection work below is still outstanding.

WHERE

Files: `src/routes/book.$slug.tsx`

Commit: a5937aa

HOW WE VERIFIED IT

Confirmed on the booking page.

WHAT REMAINS

Define a retention period with automatic deletion, add a way to export or erase a client's data on request, and reduce how much client data is sent to the AI.

SE-01 Medium — Partially fixed

Business rules lived only in the app screens

WHAT WAS FOUND

Opening hours, the booking window, cancel and move rules and revenue all lived in the browser, not on the server.

WHY IT MATTERS FOR THE BUSINESS

Anyone bypassing the app screens could ignore the salon's rules.

> **Design decision:** Owner and staff booking management (create, move, cancel) is intentionally left to the signed-in app and is not server-constrained to opening hours (see Design decisions).

WHAT WE CHANGED

The rules for public (client-facing) bookings were moved onto the server function and into the database access rules, so they hold even when the app screens are bypassed.

WHERE

Files: `drizzle/migrations/0001_audit_hardening.sql`

Commit: 28d4163, 3c08384

HOW WE VERIFIED IT

Covered by the BH-01 and SEC-03 tests, which exercise the public rules on the server.

WHAT REMAINS

Revenue and popular-services totals are still added up in the browser rather than on the server; that is tracked separately as SE-03.

SE-02 Medium

Fixed

Time slots used the visitor's time zone

WHAT WAS FOUND

The list of free time slots was calculated using the visitor's device time zone, while the salon's opening hours are stored without a time zone.

WHY IT MATTERS FOR THE BUSINESS

A client in another region saw shifted times and could book a slot that is not actually open at the salon; two clients in different regions picking the same time sent two different real times.

WHAT WE CHANGED

We compute the available time slots in the salon's time zone instead of the visitor's, and pass the salon's time zone through to the booking page.

WHERE

Files: `src/lib/format.ts`, `src/routes/book.$slug.tsx`, `drizzle/migrations/0001_audit_hardening.sql`

Commit: 28d4163

HOW WE VERIFIED IT

Test in timezone.test.ts passes in three different time zones (it produced shifted times before the change).

SE-03 Medium

Recommendation

Lists and dashboard will under-report at scale

WHAT WAS FOUND

Lists and the dashboard have no paging and silently stop at the first 1000 rows; revenue is added up in the browser.

WHY IT MATTERS FOR THE BUSINESS

As the salon grows, the dashboard and lists will quietly show incomplete or wrong totals.

HOW TO FIX

1. Calculate revenue and popular services on the server rather than in the browser.
2. Add paging to the client and booking lists so nothing is cut off at 1000 rows.

SE-04 Medium

Partially fixed

Data model: phone duplicates and no audit trail

WHAT WAS FOUND

Clients are identified by phone number with no normalisation (so '+90...', '0...' and spaced variants could create duplicates), deletes cascade, and there is no record of who changed what.

WHY IT MATTERS FOR THE BUSINESS

The same client entered two ways becomes two records, and deletions remove history with no audit trail.

WHAT WE CHANGED

We normalise phone numbers to one format before saving, so the same client typed different ways becomes a single record.

WHERE

Files: `drizzle/migrations/0001_audit_hardening.sql`

Commit: a5937aa

HOW WE VERIFIED IT

Test '[Item 12]' in public-booking.test.ts confirms two formats of the same number map to one client.

WHAT REMAINS

Switch deletes to soft-delete (hide rather than erase) and add an audit trail of who changed what and when.

SEC-07 Low

Fixed

Booking-link not cleaned on the server

WHAT WAS FOUND

A salon's public booking-link (slug) was not sanitised when set directly through the service, bypassing the app's cleanup.

WHY IT MATTERS FOR THE BUSINESS

Broken or confusing public booking links.

WHAT WE CHANGED

We clean the salon booking-link on the server and enforce a safe format, and reject negative prices as part of the same hardening.

WHERE

Files: `drizzle/migrations/0001_audit_hardening.sql`

Commit: cff6b22

HOW WE VERIFIED IT

Tests in pentest.test.ts confirm a malformed link is normalised and a negative price is rejected.

SEC-09 Low

Mitigated

AI reminder could be steered by note content

WHAT WAS FOUND

Client name and notes flow directly into the AI prompt, and the AI-written reminder is sent from the salon's own phone number.

WHY IT MATTERS FOR THE BUSINESS

Someone could try to steer the reminder message (for example to insert a link) by writing instructions into a client's note or name.

WHAT WE CHANGED

We wrap the client's name and notes as data (clearly marked as not instructions) before sending them to the AI, and strip any links out of the reminder before it can be sent.

WHERE

Files: `src/lib/ai-sanitize.ts`, `src/lib/ai.functions.ts`

Commit: 74b21bb

HOW WE VERIFIED IT

Test in ai-sanitize.test.ts removes a link smuggled into a simulated AI reply and keeps normal text. No live exploit was attempted against the production model, which is not reachable in this test setup.

WHAT REMAINS

The hard control here is stripping links from the outgoing message. Wrapping the notes as data reduces, but cannot fully guarantee against, an AI model being steered; this is why the status is 'Mitigated' rather than 'Fixed'. Keep untrusted text out of the instruction part of the prompt as the AI model changes.

SEC-11 Low

Recommendation

Environment file in the repository; dependency advisories

WHAT WAS FOUND

An environment file was committed to the generated project (it contained only the public key), and there are development-only dependency advisories.

WHY IT MATTERS FOR THE BUSINESS

Low: the committed key is public by design, and the advisories affect only the development tools, not the live app.

HOW TO FIX

1. Remove environment files from the repository and load secrets from the environment instead.
2. Update the development tool flagged by the dependency check.

SEC-12 Low

Recommendation

Password-guessing was not rate-limited (test project)

WHAT WAS FOUND

Password strength, email confirmation and protection against revealing which emails are registered were all in good shape. However, repeated wrong-password attempts were not throttled in the test project.

WHY IT MATTERS FOR THE BUSINESS

Without a limit, automated password-guessing against an account is easier.

HOW TO FIX

1. Confirm the production password and sign-in settings.
2. Enable a limit or a temporary lockout after repeated failed sign-in attempts.

SE-05 Low

Recommendation

Thin automated tests; some loosely-typed code

WHAT WAS FOUND

The generated app shipped with a single automated test and some loosely-typed code.

WHY IT MATTERS FOR THE BUSINESS

Lower confidence that future changes will not quietly break something.

HOW TO FIX

1. Add automated tests that cover the main booking and account flows.
2. Replace loosely-typed code with explicit types.
3. Use the security test suite added by this audit as a starting point.

SEC-08 Informational

Fixed

Internal helper functions callable by anyone

WHAT WAS FOUND

Two internal helper functions could be called by anonymous visitors (harmless: they return nothing useful to an anonymous caller).

WHY IT MATTERS FOR THE BUSINESS

No practical impact; addressed as defence in depth.

WHAT WE CHANGED

We removed anonymous access to the two helper functions, leaving them available only to signed-in users.

WHERE

Files: `drizzle/migrations/0001_audit_hardening.sql`

Commit: 254e527

HOW WE VERIFIED IT

Test in pentest.test.ts confirms an anonymous caller can no longer run the helper.

Remediation roadmap

Recommended order for the work still outstanding.

#	ACTION	ADDRESSES	EFFORT
1	Phone verification (one-time code) at booking	SEC-01	Medium
2	Move the login session to a secure, script-inaccessible cookie	SEC-06	Medium
3	Server-side totals and paging for lists and the dashboard	SE-03	Medium
4	Data retention, deletion and export; minimise data sent to the AI	SEC-10	Medium
5	Per-visitor rate limiting and a CAPTCHA at the edge; AI usage limit	BH-02	Low
6	Confirm production auth settings and enable password-guessing protection	SEC-12	Low
7	Keep environment files out of the repository; update development tools	SEC-11	Low
8	Broaden automated tests; tighten types; soft-delete and audit trail	SE-04, SE-05	Medium
9	Tighten the content security policy to a nonce-based one that no longer allows inline scripts	SEC-05	Low

Scope and how to re-run the tests

SCOPE

Testing was limited to a locally running copy of the app and a separate test project with the same database structure applied to it. No hosted or production environment was accessed. All test data is fictitious.

RE-RUNNING THE TESTS

The findings are backed by an automated test suite. To re-run it against your own test project, provide connection details in a local environment file (not included in this report) with these variable names:

- `AUDIT_SUPABASE_URL`
- `AUDIT_SUPABASE_PUBLISHABLE_KEY`
- `AUDIT_SUPABASE_SECRET_KEY`
- `AUDIT_DATABASE_URL`
- `AUDIT_GEMINI_API_KEY (only for the AI prompt-injection test)`

1. npm install
2. Apply the original database structure (migration 0000).
3. Apply the hardening changes (migration 0001).
4. Load the fictitious test data (fixtures).
5. Run the test suite. With only the original structure applied, the finding tests fail (proving the issue); with the hardening applied, they pass (proving the fix).